UCS503P – Software Engineering (Laboratory)

Aegis

An Encrypted Dead-Man's-Switch Legacy Vault using Shamir's Secret Sharing

Submitted by:

1024030001 Pratham Arora
1024030009 Krishna Pandey
1024030881 Nipun Behl

Group: 3C11 BE Third Year, COE

Submitted to:

Dr. Paramveer Sidhu

Prototype Evaluation (Mid-Semester)

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

2026–27 ODD Semester

Contents

1 Introduction **5**
1.1 Background and Context 5
1.1.1 Problem Statement 5
1.2 Project Objectives 6

2 Methodology and Design **7**
2.1 System Architecture 7
2.1.1 High-Level Design 7
2.1.2 Technical Stack 9
2.2 Implementation Details 14
2.2.1 Module 1: Core Functionality 14
2.2.2 Module 2: Secondary Features 16
2.2.3 Software Engineering Process 17

3 Results and Evaluation **19**
3.1 Testing Methodology 19
3.2 Performance Metrics 19
3.2.1 Demonstration Screenshots 19
3.3 Analysis 29

4 Conclusion **31**
4.1 Summary of Work 31
4.2 Key Findings 31
4.3 Future Work and Recommendations 31
4.4 Final Remarks 32

Bibliography **33**

Chapter 1

Introduction

1.1 Background and Context

A growing share of what people own and know now exists only in digital form: banking and brokerage credentials, cryptocurrency keys, password-manager master passwords, encrypted archives, business-continuity information and personal messages meant for particular people. The value of this material is entirely conditional on access, and access is exactly what is lost when its owner dies or becomes permanently incapacitated. Unlike a physical estate, a forgotten password or an unrecorded key cannot be recovered by an executor; the data simply becomes unreachable.

Existing approaches force an uncomfortable choice. Sharing passwords with a relative or friend makes the secret readable from the moment it is shared, and its safety then depends indefinitely on the honesty, competence and availability of one person. Appointing a single executor or custodian concentrates the same risk in one party and is triggered by a slow legal process rather than by the owner's absence. Provider-held recovery schemes, in which a service keeps the means to unlock an account, require trusting the provider and anyone who compromises it with the plaintext. In every case there is a single point of failure: one document, one person or one operator whose loss, compromise or delay defeats the owner's intent.

The project therefore combines two ideas. A *dead man's switch* releases information when its owner stops responding, so the trigger is the owner's sustained silence rather than a human decision. *Threshold secret sharing* [Sha79] removes the single custodian: the decryption key is split among several trustees so that any K of N of them can rebuild it, while any $K - 1$ learn nothing at all. Aegis adds a third property that recent breaches have made essential: the service that runs the switch should never be able to read what it protects.

1.1.1 Problem Statement

The problem addressed is how to release a secret to designated recipients when, and only when, its owner has become unresponsive, without any single party — including the system operator — being able to read the secret beforehand, and without depending on any single custodian. The scope of the prototype is fixed by the SRS: one vault per owner (`NG-3`); check-in prompts and release notices by email only, with no SMS (`NG-1`) and no native mobile application (`NG-2`); N trustees with a reconstruction threshold of at least two (`FR-4`); and a zero-knowledge server that stores only ciphertext and trustee-encrypted shares (`NFR-SEC-5`). The catastrophic failure mode is premature release, which is irreversible (`NFR-REL-1`).

1.2 Project Objectives

1. **Objective 1:** Never release a vault before its deadline plus grace period: zero early releases across at least 1,000 simulated schedules with injected faults, by Week 8 (`NFR-REL-1`, `NFR-REL-3`).
2. **Objective 2:** Keep the server zero-knowledge: an automated inspection of everything the server stores and receives finds no plaintext payload, key or share, by Week 8 (`FR-2`, `FR-2a`, `NFR-SEC-1`, `NFR-SEC-5`).
3. **Objective 3:** Make the threshold property demonstrable and verified: any K of N shares recover the vault, fewer than K fail, verified by property-based tests and an end-to-end browser test of the full demo script, by Week 8 (`FR-8`, `NFR-SEC-2`).
4. **Objective 4:** Implement the cryptographic core twice — an authoritative Python implementation and the deployed TypeScript client — and bind them with shared known-answer test vectors that agree on every case (`FR-2a`).
5. **Objective 5:** Keep check-ins fast and the code maintainable: check-in confirmation p95 within 500 ms under nominal load (`NFR-PERF-1`) and at least 80% test coverage on the crypto and scheduler packages (`NFR-MAINT-1`), deployed on Vercel for the prototype demonstration.

The prototype was built to these SMART objectives (Specific, Measurable, Achievable, Relevant, Time-bound), each tied to SRS requirements and to the prototype milestone (Week 8 of the 12-week plan).

Chapter 2

Methodology and Design

2.1 System Architecture

Aegis is a web application with a React single-page frontend and a FastAPI backend, deployed as a single Vercel project so that the browser talks to one origin and the session cookie needs no cross-origin configuration [Ver]. All payload cryptography runs in the browser; the server schedules check-ins, stores ciphertext and encrypted shares in Postgres, and sends email. Because serverless functions do not keep a process alive, the scheduler is driven by external triggers that call an idempotent tick endpoint. Figure 2.1 shows the deployment and Figure 2.2 the client/server trust boundary.

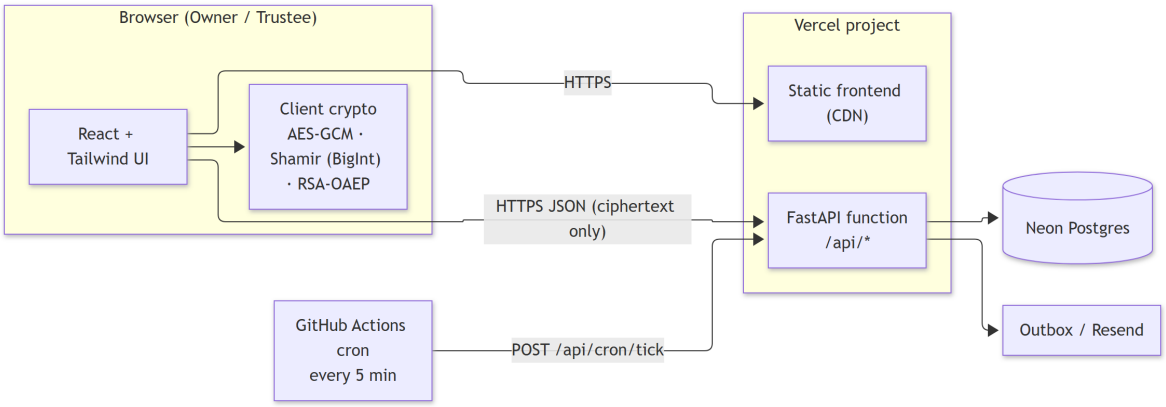


Figure 2.1: Deployment architecture of the prototype

2.1.1 High-Level Design

Components. The browser hosts the user interface and the client crypto module (AES-256-GCM, a hand-written Shamir implementation over `BigInt`, and RSA-OAEP). The FastAPI application exposes authentication, vault, check-in, trustee, cron and demo endpoints under `/api` [Ram]. Four domain packages sit behind it: `crypto` (the authoritative specification and test oracle), `scheduler` (the lifecycle), `vault` (SQLAlchemy models and repositories [SQL]) and `notifications` (a transactional outbox).

Trust model. The server is trusted for scheduling and delivery, never for confidentiality. The browser generates a random 256-bit key, encrypts the payload, splits the key into N shares and encrypts each share to its trustee's RSA public key; the server receives only the results. Trustees generate their keypairs in their own browsers and keep the private key as a downloaded file. A server breach alone therefore reveals

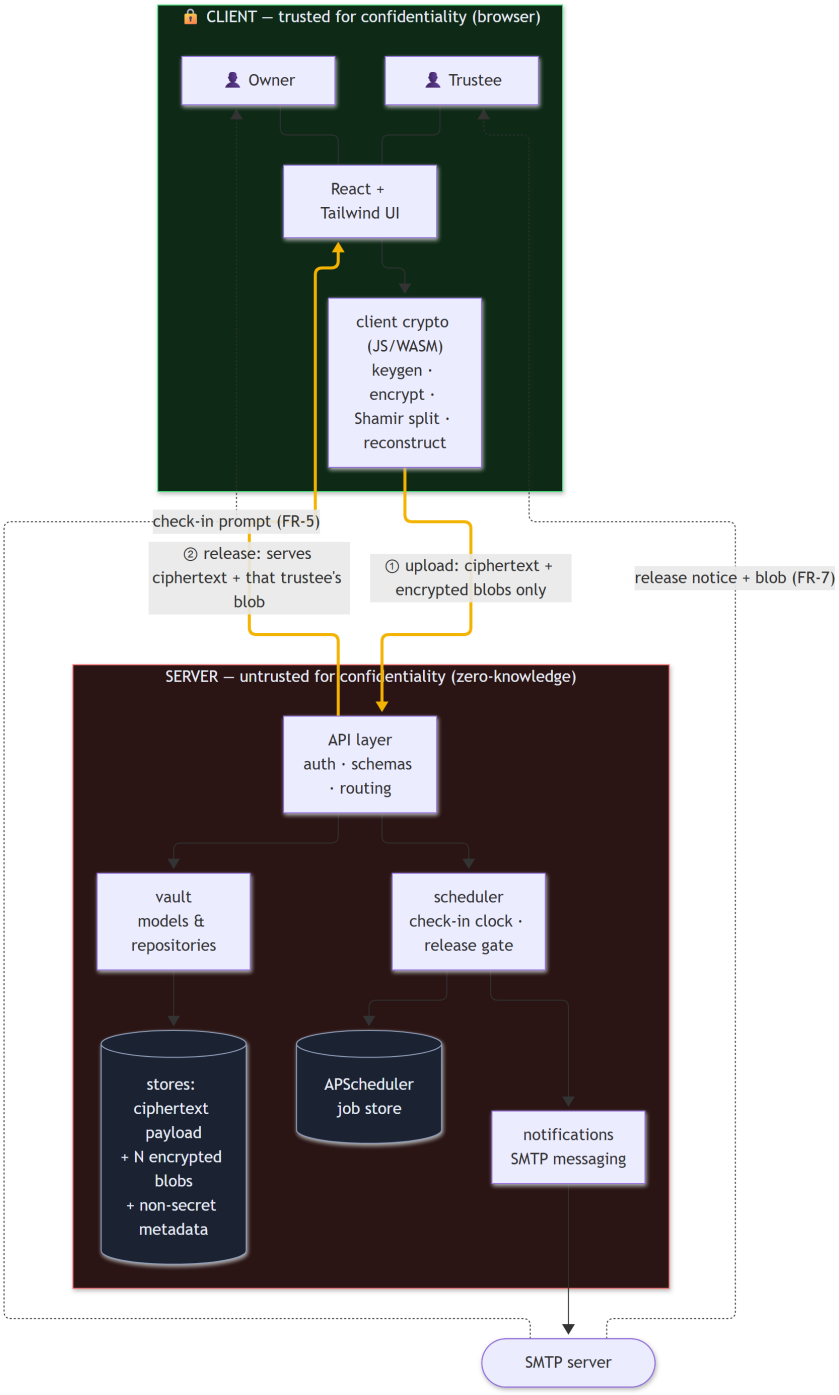


Figure 2.2: Client/server trust boundary: only ciphertext and encrypted blobs cross it

nothing; a breach of confidentiality requires the server *and* at least K trustees. The documented residual risks (for example, a malicious operator serving altered client code) are discussed in Section 3.3.

Vault state machine. Each vault moves through Setup, Active, Warning, Grace and Released (Figure 2.3). The decision is made by a single pure function, `evaluate(clock, now)`, which releases a vault only when the current time has reached the deadline plus the full grace period with no confirmed check-in. A check-in from Active, Warning or Grace returns the vault to Active; Released is terminal.

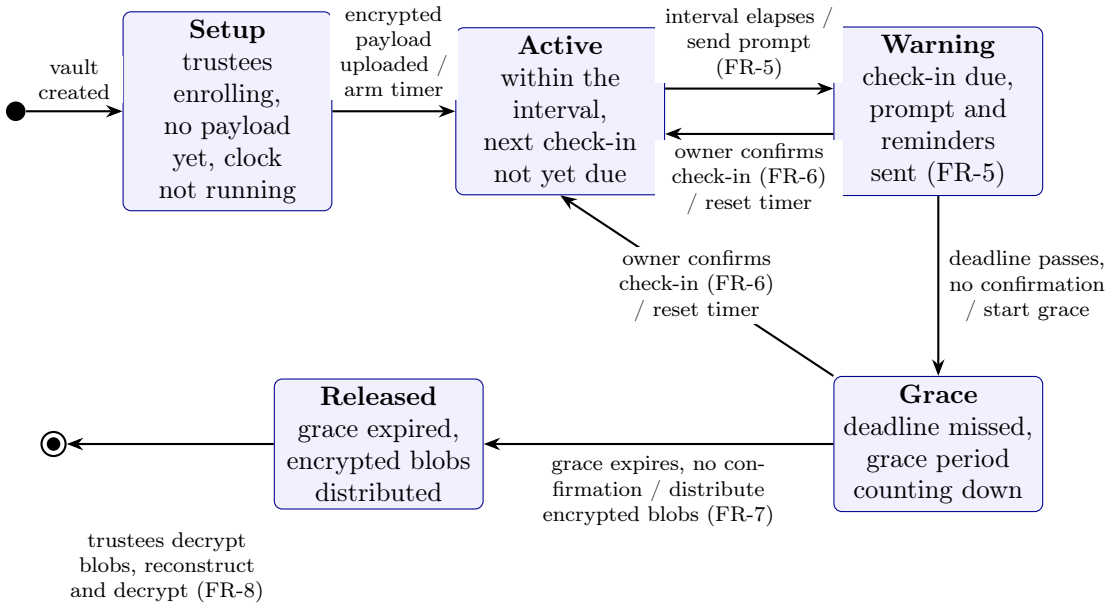


Figure 2.3: Vault lifecycle state machine with its triggers and actions

Figures 2.4 and 2.5 show the two principal flows: creating and sealing a vault, and releasing and recovering it. Figure 2.6 shows the data model; the use-case view is Figure 2.7.

2.1.2 Technical Stack

- **Framework/Language:** Python 3.12 with FastAPI for the API; TypeScript with React 18, Vite and Tailwind CSS for the client [Met].
- **Database:** SQLAlchemy 2 with SQLite for local development and tests, and Postgres on Neon in deployment (`NFR-PORT-1`). Payloads are stored as ciphertext in the database, capped at about 1 MB.
- **Tools:** Git and GitHub with a `prototype` branch; GitHub Actions for continuous integration and the 5-minute scheduler trigger; Vercel for hosting; Playwright for end-to-end testing [Mic]; ruff, ESLint and Prettier for code quality.
- **Libraries:** the browser's Web Cryptography API for AES-256-GCM [Dwo07] and RSA-OAEP-2048 with SHA-256 [Mor+16; Wat17]; `argon2-cffi` for pass-

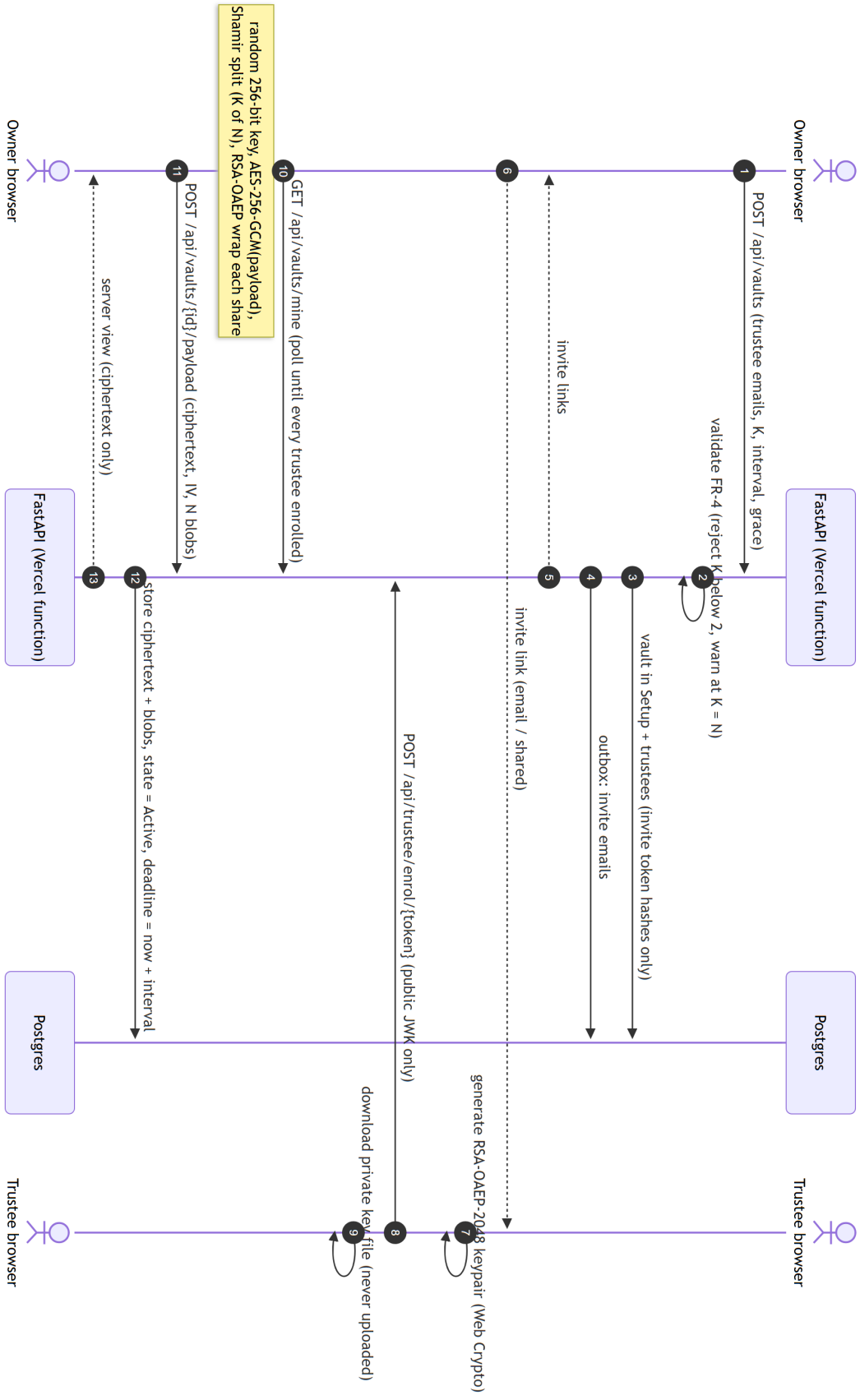


Figure 2.4: Sequence: creating and sealing a vault

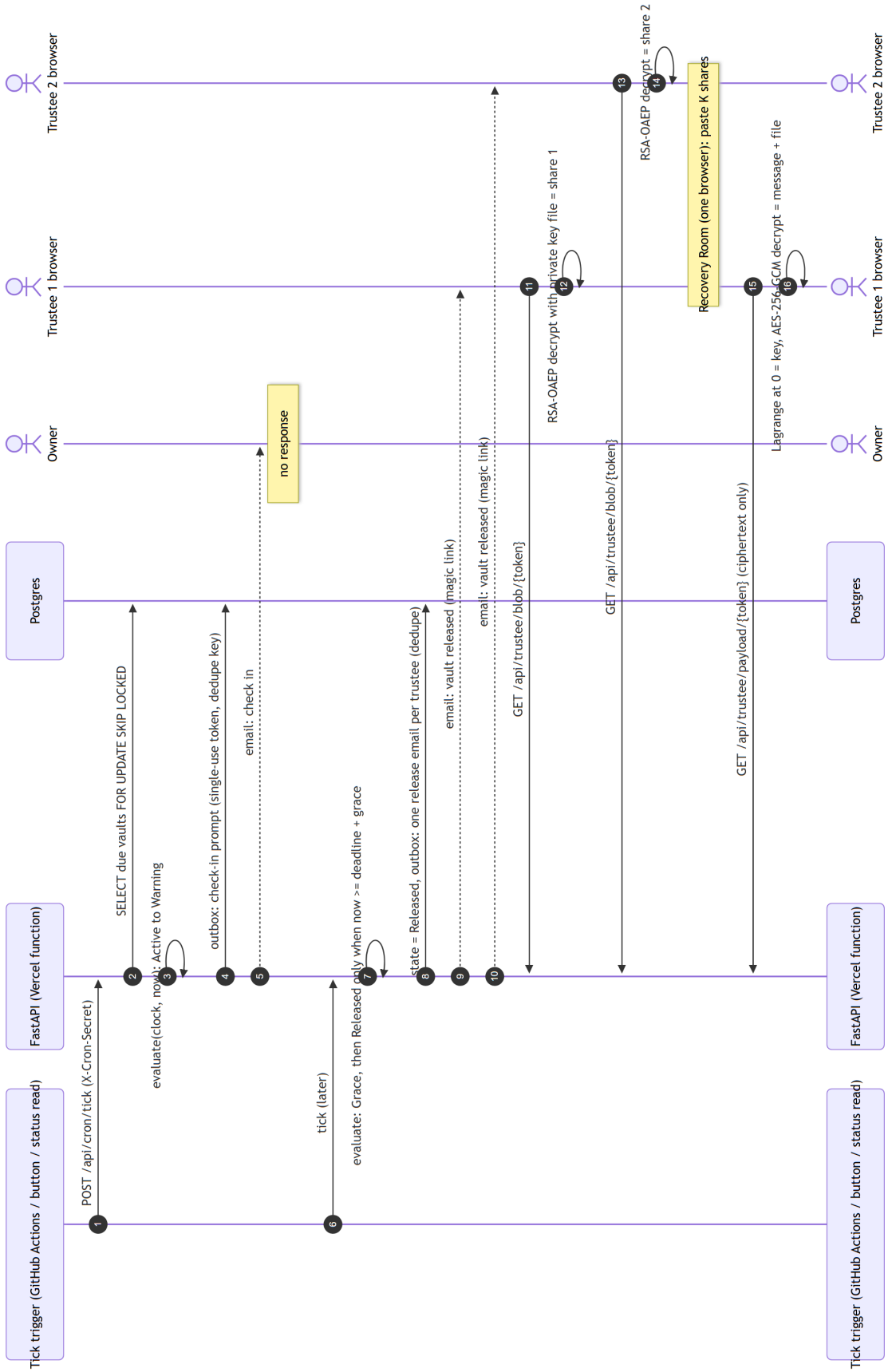


Figure 2.5: Sequence: release and K-of-N recovery

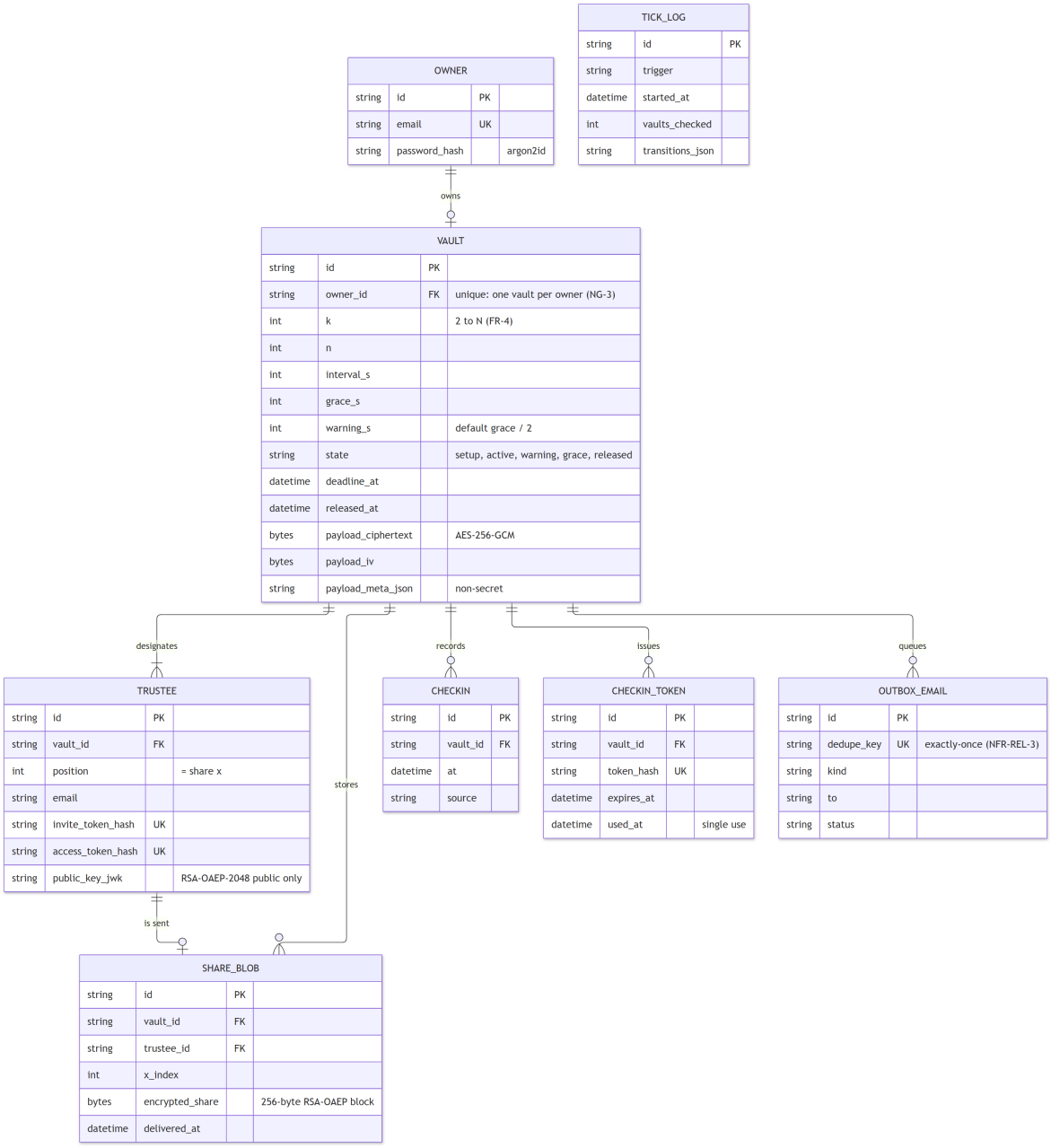


Figure 2.6: Data model (SQLAlchemy entities)

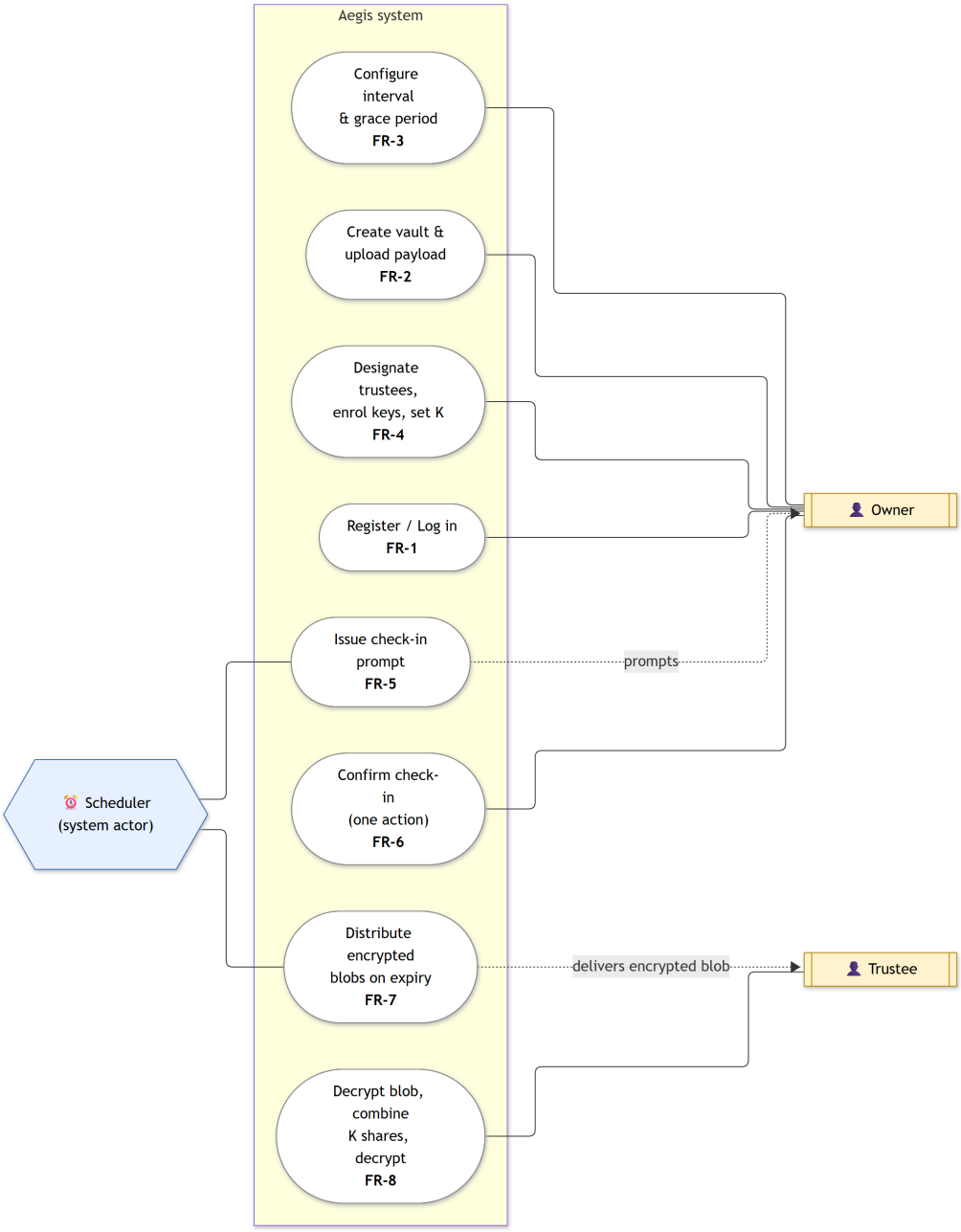


Figure 2.7: Use-case diagram: Owner, Trustee and the time-triggered Scheduler

word hashing [Bir+21]; PyJWT for session tokens; pytest, Hypothesis [MH+19] and Vitest for testing.

Table 2.1 summarises each technology choice and the reason for it.

Table 2.1: Technology choices and their reasons

Layer	**Technology**	**Reason**
Frontend	React 18 + TypeScript + Vite + Tailwind	Planned stack; static build served from Vercel's CDN
Client crypto	Web Crypto (AES-256-GCM, RSA-OAEP-2048/SHA-256)	Built into browsers; no third-party crypto library on the client
Secret sharing	Hand-written Shamir over $\mathrm{GF}(2^{521}-1)$	SRS constraint; Python and TypeScript versions bound by shared test vectors
Backend	Python 3.12 + FastAPI as a Vercel Python function	Planned stack; same origin as the frontend
Database	SQLAlchemy 2; SQLite locally, Neon Postgres deployed	`NFR-PORT-1`; serverless functions have no persistent disk
Scheduler	Pure `evaluate()` + idempotent `POST /api/cron/tick`	Serverless functions keep no process alive, so no APScheduler
Tick triggers	GitHub Actions (5 min), Demo Console button, lazy due-checks, daily Vercel Cron	Hobby-plan Vercel Cron runs at most once a day
Email	Outbox + `EmailSender` (log / Resend)	Demo works without a verified email domain
Auth	argon2id + JWT in an httpOnly, SameSite=Lax cookie	`NFR-SEC-3`
Testing	pytest, Hypothesis, pytest-cov, Vitest, Playwright	`NFR-MAINT-1` and the demo script as a test

2.2 Implementation Details

2.2.1 Module 1: Core Functionality — Client-Side Encryption and Shamir's Secret Sharing

Finite field. All arithmetic is carried out modulo the Mersenne prime $p = 2^{521} - 1$, which is large enough to hold a 256-bit AES key as a single field element. Division is multiplication by a modular inverse (Fermat's little theorem in TypeScript, Python's three-argument `pow` in the oracle).

Polynomial construction. To split a key S with threshold K, the client draws $K - 1$ coefficients uniformly at random from the field and forms

$$f(x) = S + a_1 x + a_2 x^2 + \cdots + a_{K-1} x^{K-1} \pmod{p},$$

so that $f(0) = S$. Trustee i receives the point $\big(i, f(i)\big)$ for $i = 1, \ldots, N$; $x = 0$ is never used because $f(0)$ is the secret.

Reconstruction. Any K points determine a unique polynomial of degree at most $K - 1$, so the secret is recovered by Lagrange interpolation evaluated at zero:

$$S \;=\; \sum_{i} y_i \prod_{j \neq i} \frac{x_j}{x_j - x_i} \pmod{p}. \tag{2.1}$$

With only $K-1$ points, every candidate secret is consistent with exactly one polynomial through those points, so the shares reveal nothing about S [Sha79]. A Hypothesis property test checks this directly: for random $K - 1$ shares and a random candidate secret, it constructs coefficients that the real split code turns into exactly those shares.

Share encoding and wrapping. A share is encoded as 68 bytes (a version byte, the x-coordinate and a 66-byte y) and encrypted to its trustee's RSA-OAEP-2048 public key, producing a 256-byte blob; this fits RSA-OAEP-SHA-256's 190-byte plaintext limit. Shares that trustees exchange in the Recovery Room use the text form `aegis-share:v1:<x>:<y-base64url>`. The payload is encrypted with AES-256-GCM using the vault identifier as associated data, so a ciphertext cannot be opened as another vault's.

Two implementations, one specification. The Python package `code/crypto` is the authoritative specification and test oracle; it emits a versioned file of known-answer vectors (Shamir splits from known coefficients, reconstructions, AES-GCM and RSA-OAEP cases). The TypeScript client must reproduce every vector, and continuous integration fails if the two disagree. Result: 13/13 matched.

Algorithms 1 and 2 give the client-side sealing and recovery steps.

Algorithm 1 SEAL: client-side sealing of a vault

function SEAL($payload, trustees, K$)
  $key \leftarrow$ 32 random bytes; $iv \leftarrow$ 12 random bytes
  $aad \leftarrow$ 'aegis-vault:v1:' $\|$ $vault_id$
  $ct \leftarrow$ AES-256-GCM-ENCRYPT($key, iv, payload, aad$)
  $a_1, \ldots, a_{K-1} \leftarrow$ uniform random field elements
  for $i \leftarrow 1$ **to** N **do**
    $share_i \leftarrow \big(i, f(i)\big)$ where $f(x) = \mathrm{int}(key) + a_1 x + \cdots + a_{K-1} x^{K-1} \bmod p$
    $blob_i \leftarrow$ RSA-OAEP-ENCRYPT($trustee_i.public_key$, ENCODE($share_i$))
  end for
  UPLOAD($ct, iv, blob_1, \ldots, blob_N$); WIPE($key$)
end function

Algorithm 2 RECOVER: K-of-N recovery in the Recovery Room

function RECOVER($shares, K, ct, iv$)
  if $|shares| < K$ **then**
    return report 'not enough shares'
  end if
  $S \leftarrow \sum_i y_i \prod_{j \neq i} x_j/(x_j - x_i) \bmod p$
  $key \leftarrow$ 32-byte big-endian encoding of S
    (fails if S does not fit: wrong or too few shares)
  return AES-256-GCM-DECRYPT(key, iv, ct, aad)
    (GCM authentication fails on any error)
end function

Worked example (toy prime $p = 31$). Take secret $S = 13$, threshold $K = 3$ and $N = 5$, with random coefficients $a_1 = 7$ and $a_2 = 4$, so

$$f(x) = 13 + 7x + 4x^2 \pmod{31}.$$

The five shares are $(1, 24)$, $(2, 12)$, $(3, 8)$, $(4, 12)$, $(5, 24)$. Using the shares at $x = 1, 3, 5$, the Lagrange basis values at zero are $\ell_1(0) = 29$, $\ell_2(0) = 22$, $\ell_3(0) = 12 \pmod{31}$, and

$$\sum_i y_i \, \ell_i(0) = 24 \cdot 29 + 8 \cdot 22 + 24 \cdot 12 \equiv 13 \pmod{31},$$

which is the secret.

With only two shares, $(1, 24)$ and $(3, 8)$, every candidate secret fits some quadratic: for example $S = 0$ gives $f(x) = 0 + 14x + 10x^2$; $S = 5$ gives $f(x) = 5 + 28x + 22x^2$; $S = 22$ gives $f(x) = 22 + 26x + 7x^2$ (all mod 31), each passing through both known shares. Two shares therefore say nothing about S.

2.2.2 Module 2: Secondary Features

Scheduler and tick design. The lifecycle is decided by the pure function `evaluate(clock, now)`, which returns the new state and the actions owed on the way (check-in prompt, grace reminder, release). It moves only forward, so a backward clock jump never undoes a transition, and it catches up through missed transitions in order after an outage. The SRS fixes only the interval and grace period (`FR-3`); the prototype places the Warning-to-Grace boundary inside the grace window (by default at its midpoint), so the release instant remains exactly the deadline plus the grace period. A late check-in, arriving at or after that instant, is refused, so the outcome never depends on whether a tick happened to run first.

Transactional tick and exactly-once release. `tick()` loads due vaults with `SELECT ... FOR UPDATE SKIP LOCKED` on Postgres (SQLite takes the write lock up front), applies `evaluate` to the locked rows and writes the state change and the outbox rows in one transaction. Each outbox row has a unique deduplication key, for example `release:<vault>:<trustee>`, so a release is recorded exactly once even when two ticks race (`NFR-REL-3`). Deadlines live in the database, so a restarted process resumes

where the previous one stopped (NFR-REL-2). Ticks come from GitHub Actions every five minutes, a Demo Console button, a cheap due-check whenever an owner or trustee status endpoint is read, and a daily Vercel Cron backstop.

Check-in tokens. Prompt emails carry a one-click link with a single-use, expiring token; only its SHA-256 hash is stored. The link's GET request only previews the token; the page confirms with a POST, so email security scanners that pre-fetch links cannot check in on the owner's behalf (threat T-5).

Trustee enrolment. Each trustee opens an invitation link; their browser generates an RSA-OAEP-2048 keypair, uploads only the public key and forces a download of the private-key file (optionally also kept in IndexedDB). The API rejects any key containing private-key fields. FR-4 is enforced on the server and in the user interface: K below 2 is rejected, $K = N$ produces a warning that losing any trustee makes the vault unrecoverable, and the loss tolerance $N - K$ is displayed.

Notification service. Email is a strategy behind an `EmailSender` interface: `log` mode records every message in an in-app Outbox (used in the demonstration), and `resend` mode also sends through Resend.

Frontend pages. A landing page; registration and login; an owner dashboard with a live countdown on the server's clock and a single "I'm OK — check in" button; a vault wizard that asks for the secret only after every trustee has enrolled and then shows exactly what the server received; the one-click check-in page; trustee enrolment and portal pages; a Recovery Room that reconstructs and decrypts entirely in the browser; and a token-protected Demo Console with the Outbox, tick log, server view and measured results.

Deployment. One Vercel project builds the Vite frontend as static files and runs `api/index.py` as a Python function; rewrites send `/api/*` to the function and all other paths to the single-page app.

2.2.3 Software Engineering Process

The project follows the incremental 12-week plan in `docs/plan/twelve-week-plan.md`: requirements and scaffolding (Week 1), the zero-knowledge trust model and threat model (Week 2), and the prototype (Week 8) [PM20]. Work is done on feature branches with small conventional commits; the prototype was built on a `prototype` branch and merged through a pull request. Continuous integration runs ruff, pytest with a coverage gate, ESLint, Vitest (including the cross-language vectors), the production build and the end-to-end test on every push. Requirements are traced to modules and tests as shown in Table 2.2.

Table 2.2: Requirement traceability: requirement → module → test

Requirement	**Module**	**Verified by**
FR-1, NFR-SEC-3	`code/api/routes/auth.py`, `security.py`	`test_api.py` (register/login, argon2id hash, cookie flags)
FR-2, FR-2a, NFR-SEC-1, NFR-SEC-5	`frontend/src/crypto/vault.ts`	`vault.test.ts`; `inspect_server_store.py`
FR-3, FR-4	`code/api/routes/vaults.py`; `VaultNew.tsx`	`test_api.py` ($K < 2$, $K = N$, interval); E2E step 3
FR-5, FR-7	`code/scheduler/state.py`, `tick.py`	`test_scheduler_state.py`, `test_scheduler_tick.py`
FR-6, NFR-USE-1	`code/api/routes/checkin.py`; `CheckinLink.tsx`	`test_api.py`; E2E (emailed link)
FR-8, NFR-SEC-2	`shamir.py` / `shamir.ts`; `Recover.tsx`	Hypothesis properties; `vault.test.ts`; E2E steps 7–8
NFR-REL-1/2/3	`code/scheduler`	`reliability_sim.py` (1,000 schedules)
NFR-PERF-1	`code/api`	`load_checkin.py`
NFR-MAINT-1	`code/crypto`, `code/scheduler`	`pytest -cov` (CI gate at 80%)
NFR-PORT-1	`code/vault/db.py`	SQLite in tests; Neon Postgres in deployment

Chapter 3

Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** pytest for the Python packages and Vitest for the TypeScript crypto. Results: 125/125 passed (Python) and 22/22 passed (TypeScript). Property-based testing with Hypothesis [MH+19] covered 6 properties with 1600 generated cases and 0 failures.
- **Integration Testing:** FastAPI's test client drives the full demo script server-side with the Python crypto acting as the browser; the Python and TypeScript implementations are bound by the shared vectors (13/13 matched); and a Playwright test runs the whole demo script through the real user interface in separate browser contexts for the owner and each trustee (passed).
- **Performance Testing:** a reliability simulator runs the real scheduler code against 1,000 schedules with a fake clock and injected faults; a load script measures check-in latency; a timing harness measures the client crypto operations.
- **User Testing:** limited to the team. The three team members used the prototype themselves, following the demo script; no testing with users outside the team has been done yet, so this report has no external usability findings. A walkthrough with classmates remains to be done.

3.2 Performance Metrics

Table 3.1 lists every measured result against its target.

Environment and method notes. Reliability: 1000 schedules; 17250 ticks including 1721 duplicate and 837 racing pairs, 350 restarts, clock jitter ±3 s, 294 too-late check-ins refused. The local latency was measured against uvicorn with SQLite on a Windows laptop. Crypto timings were measured with Node.js v24.21.0 Web Crypto (win32/x64), not in a browser. Every value above can be regenerated with `python scripts/collect_metrics.py`.

3.2.1 Demonstration Screenshots

Figures 3.1 to 3.11 show the prototype at each step of the demonstration script.

Table 3.1: Prototype performance results

Metric	**Target**	**Achieved**
Early releases, 1,000 schedules (NFR-REL-1)	0	0 (requirement cadence); 0 (cron only)
Duplicate releases (NFR-REL-3)	0	0; 0
Missed releases (FR-7)	0	0; 0
Max release lateness (NFR-REL-2)	≤ 60 s	56.349 s (met); 809.195 s with the 5-minute cron alone (**not met**)
Max prompt dispatch delay (NFR-PERF-2)	≤ 60 s	51.911 s (met); 828.542 s cron only (**not met**)
Check-in latency p95, local (NFR-PERF-1)	≤ 500 ms	23.7 ms sequential; 1079.0 ms for 50 simultaneous requests on one vault
Check-in latency p95, deployed (NFR-PERF-1)	≤ 500 ms	352.3 ms sequential (first request 295.8 ms); 2918.9 ms for 50 simultaneous requests on one vault
Zero-knowledge inspection findings (NFR-SEC-1/5)	0	0
Coverage: crypto (NFR-MAINT-1)	$\geq 80\%$	100.0%
Coverage: scheduler (NFR-MAINT-1)	$\geq 80\%$	98.8%
Test pass rate	100%	125/125 passed; 22/22 passed
Cross-language vectors	all	13/13 matched
Crypto timing: AES-256-GCM encrypt, 1 MiB	—	median 0.922 ms, p95 1.179 ms
Crypto timing: AES-256-GCM decrypt, 1 MiB	—	median 0.814 ms, p95 1.216 ms
Crypto timing: RSA-OAEP wrap, one share	—	median 0.124 ms, p95 0.234 ms
Crypto timing: RSA-2048 keypair generation	—	median 39.32 ms, p95 126.587 ms
Crypto timing: Shamir split (3, 5)	—	median 0.016 ms, p95 0.028 ms
Crypto timing: Shamir reconstruct (3, 5)	—	median 0.504 ms, p95 0.606 ms
Crypto timing: Shamir reconstruct (5, 7)	—	median 2.514 ms, p95 2.658 ms
Lighthouse, landing / dashboard (mobile profile)	—	Performance 100, Accessibility 100, Best practices 96, SEO 91 (both pages)

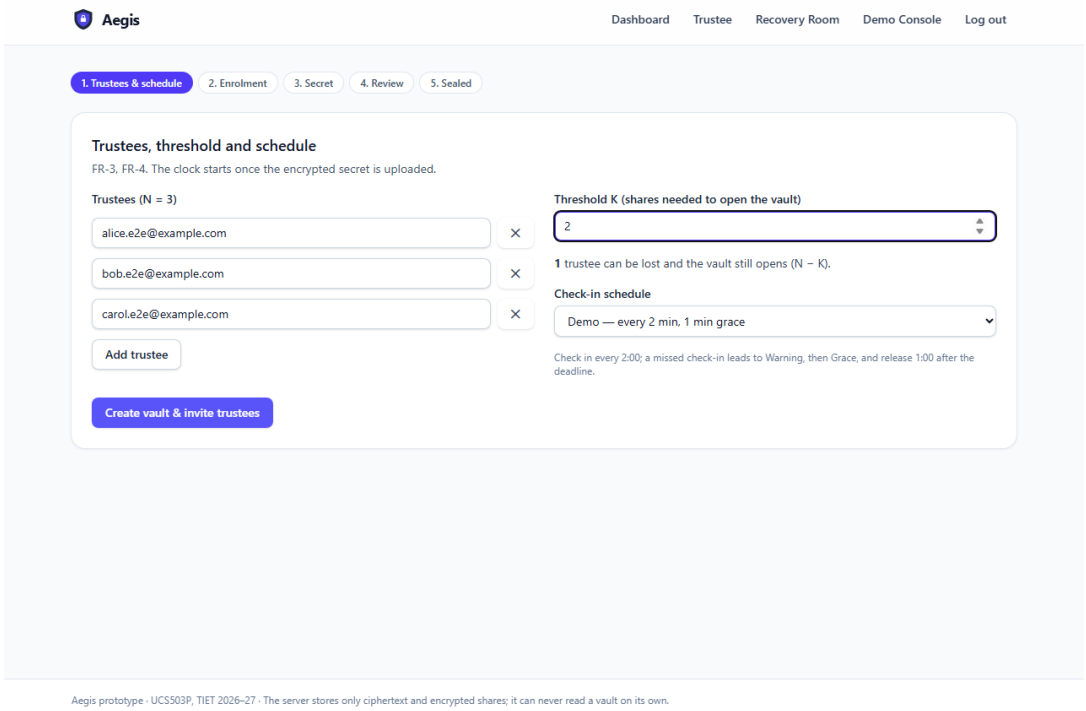


Figure 3.1: Vault wizard: trustees, threshold $K = 2$ of $N = 3$ and loss tolerance

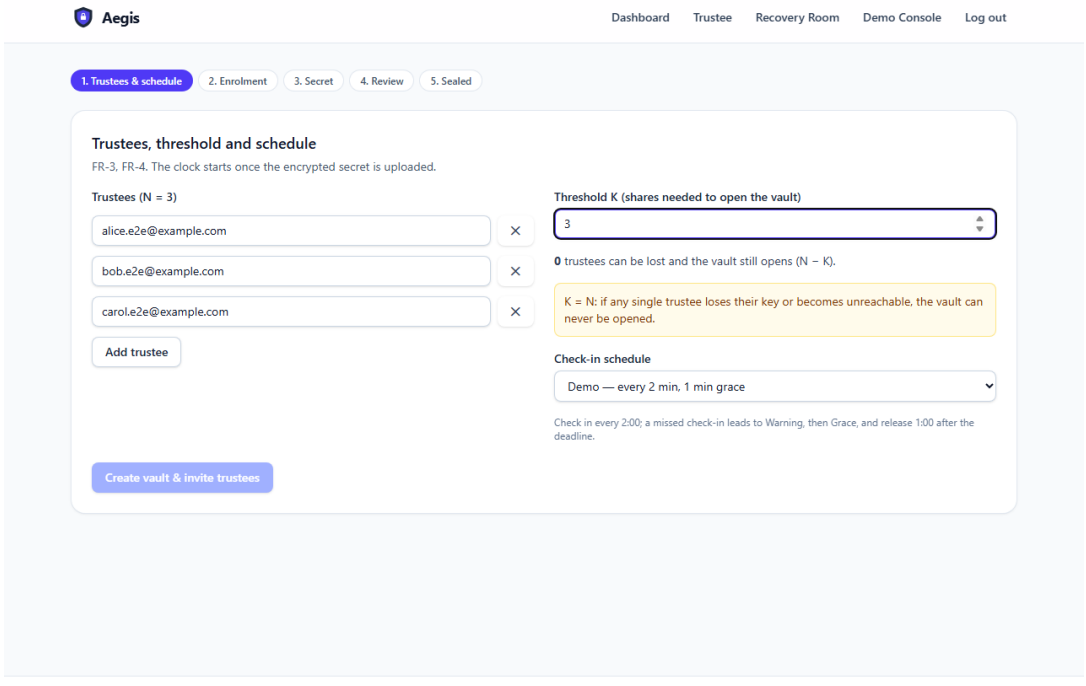


Figure 3.2: FR-4 validation: $K = N$ produces a warning ($K < 2$ is rejected)

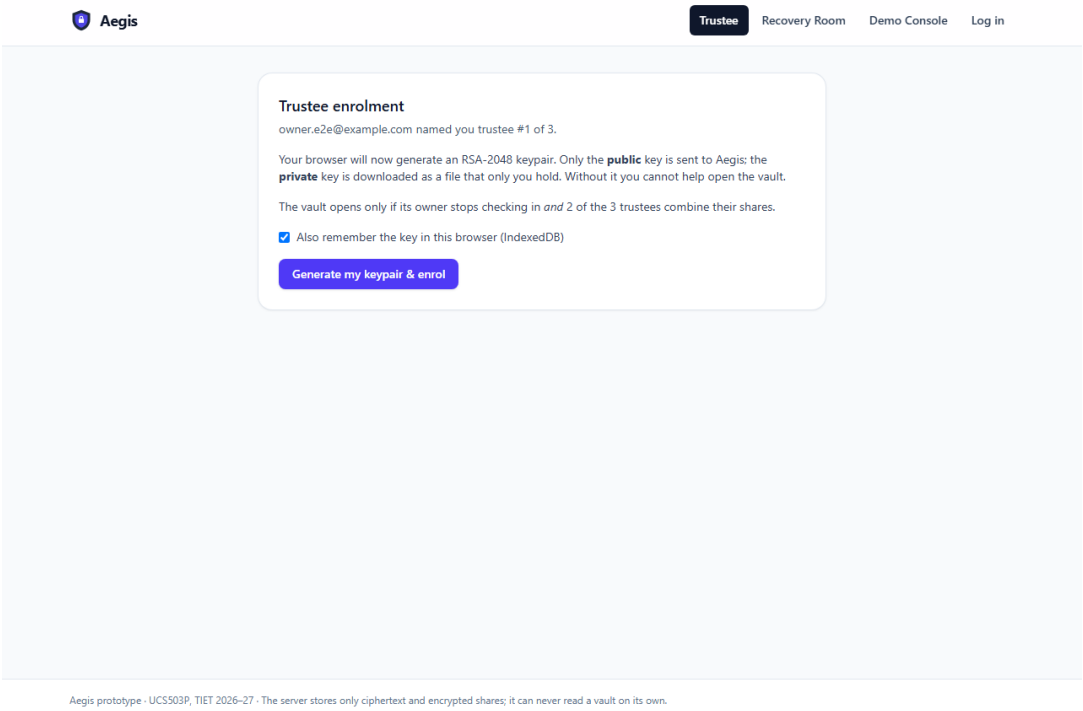


Figure 3.3: Trustee enrolment: the keypair is generated in the trustee's browser



Figure 3.4: What the server received and stores: ciphertext and encrypted shares only

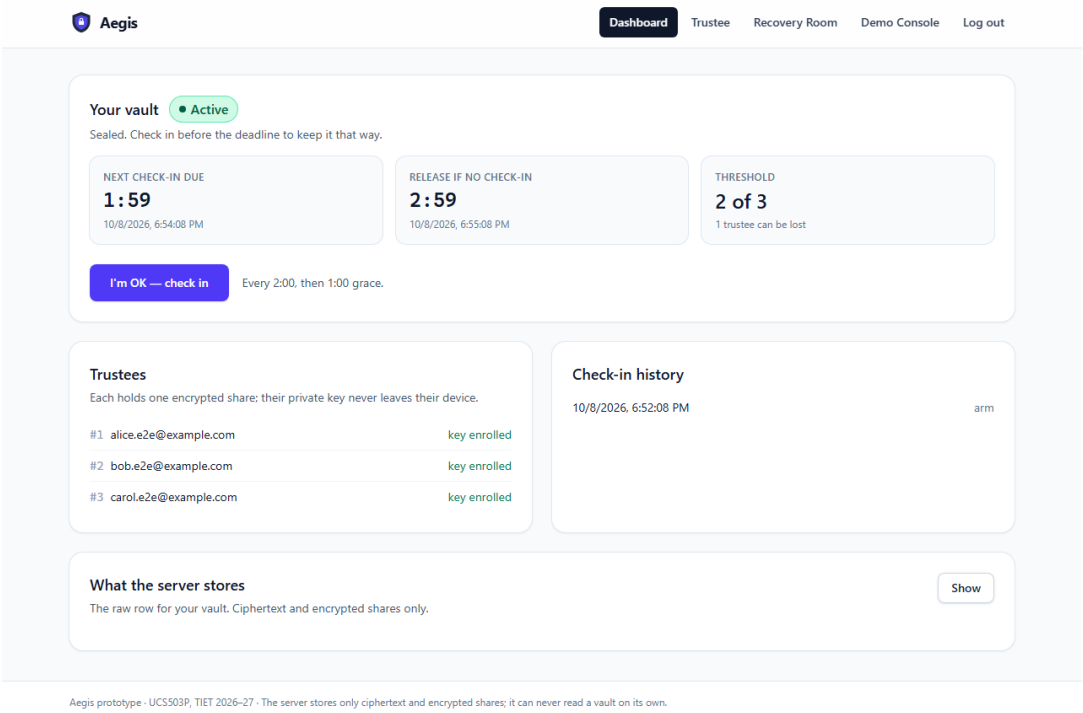


Figure 3.5: Owner dashboard: Active state, live countdowns and one-click check-in

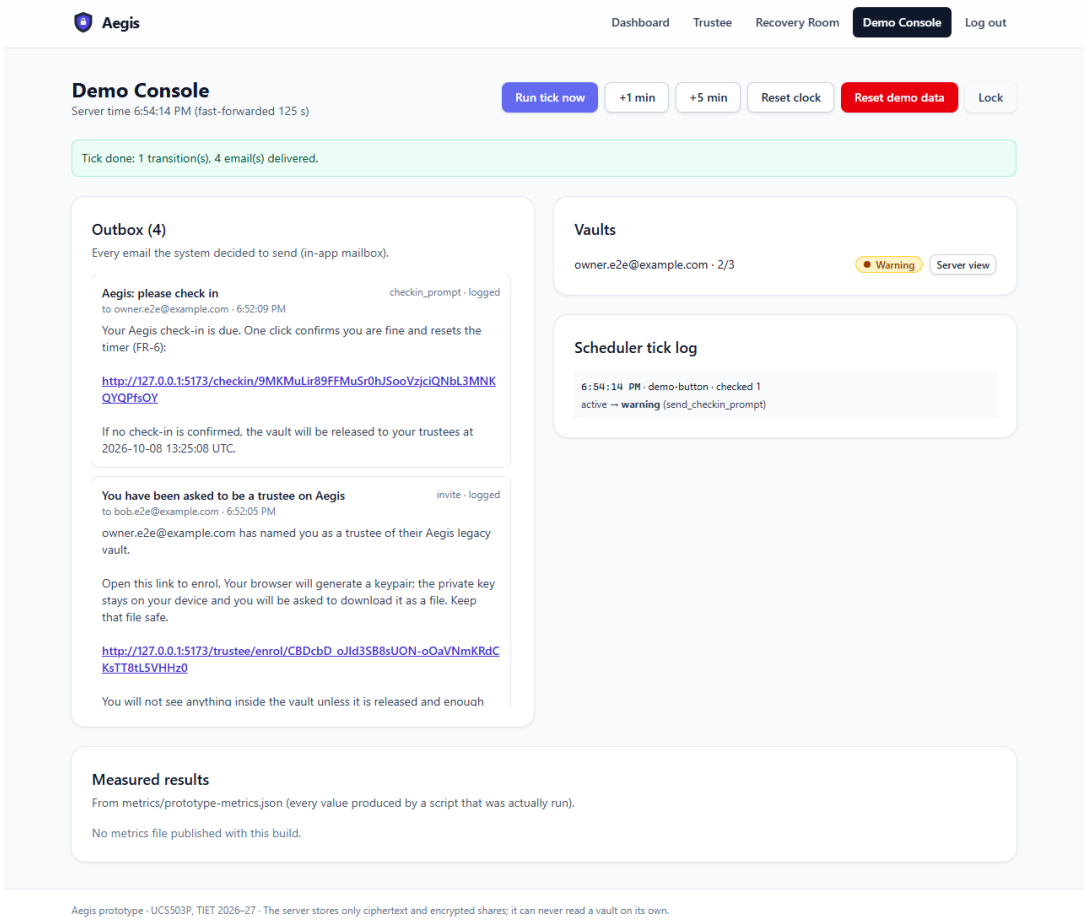


Figure 3.6: Demo Console after a tick: Warning state and the check-in prompt in the Outbox

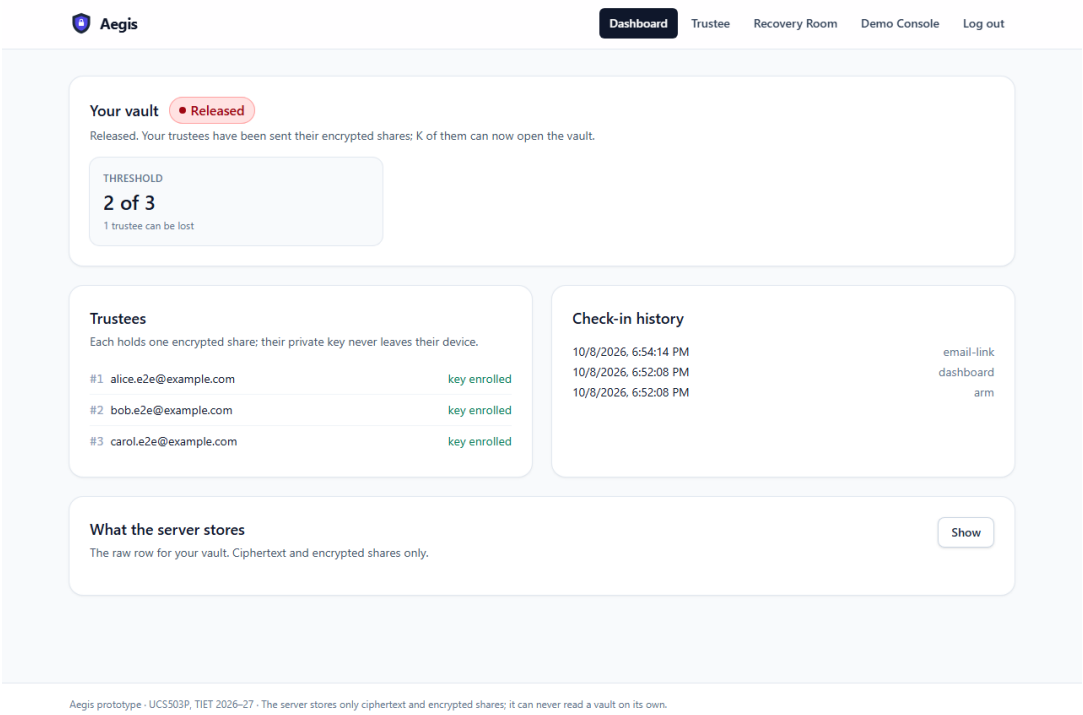


Figure 3.7: The vault after the grace period expired: Released

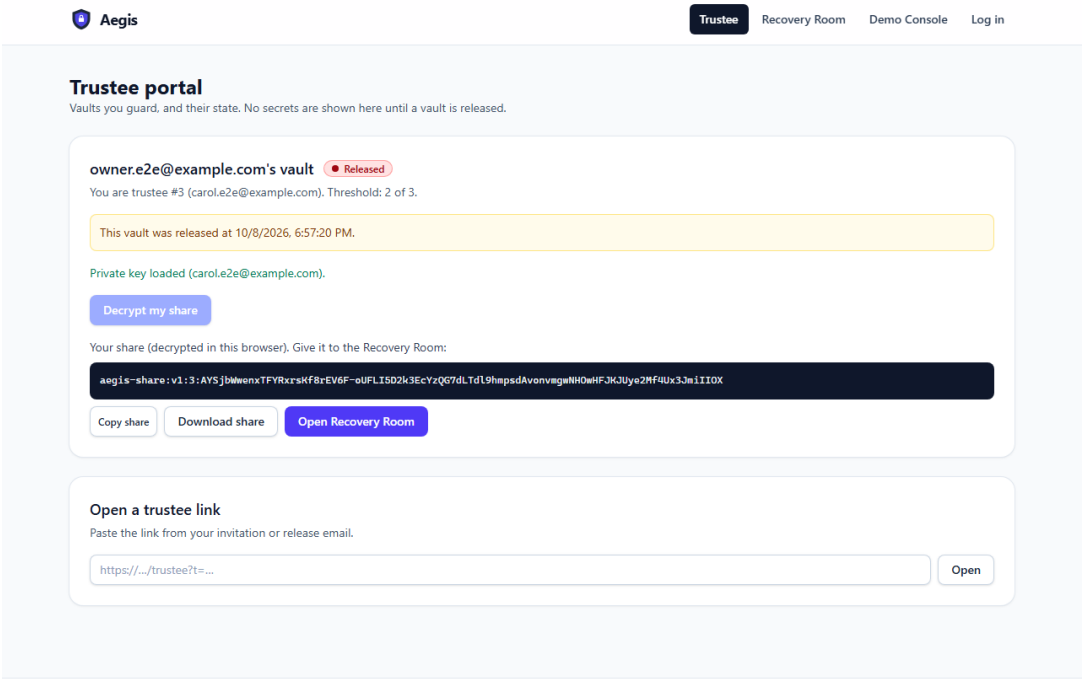


Figure 3.8: Trustee portal: share decrypted in the browser with the private key file

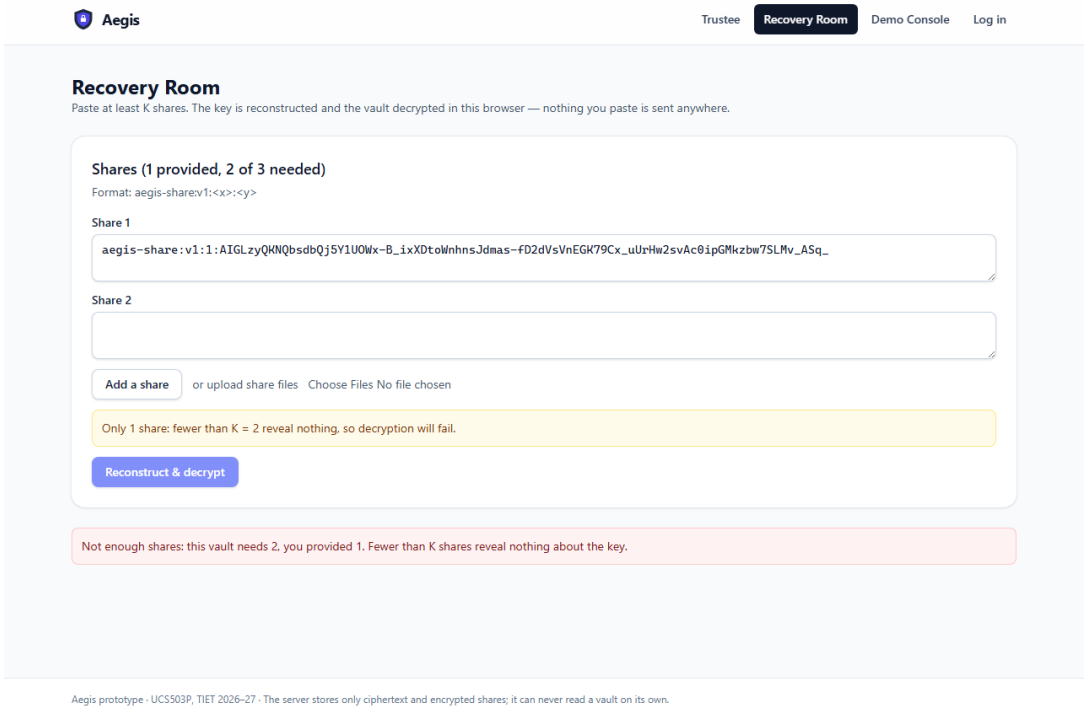


Figure 3.9: Recovery Room: one share alone is refused

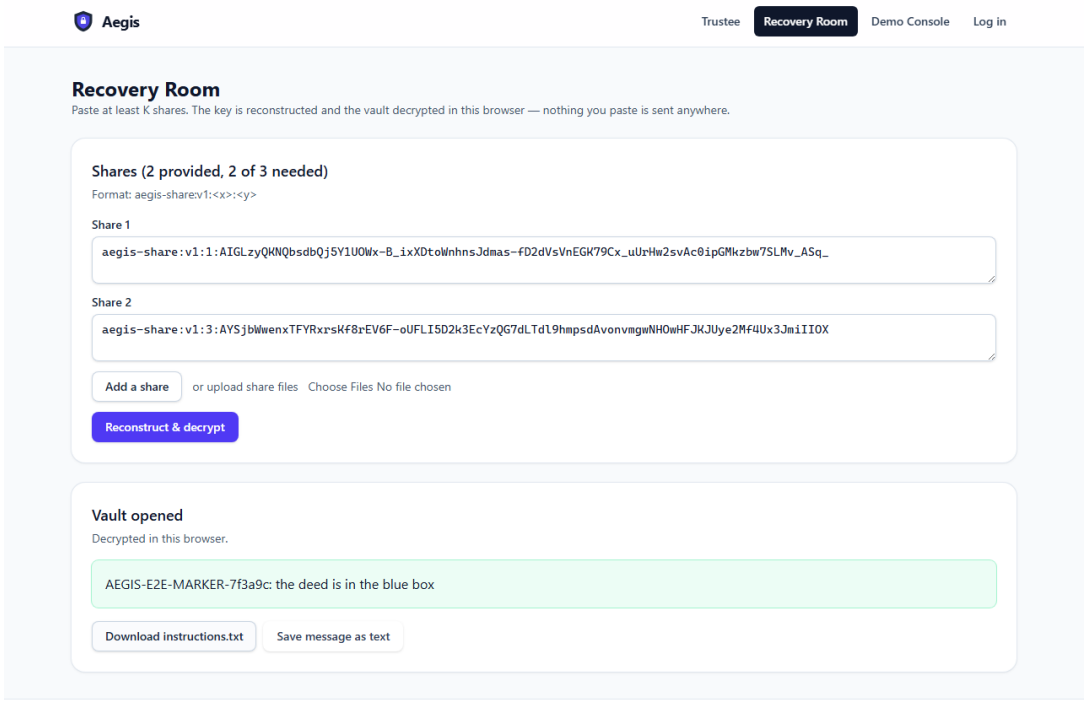


Figure 3.10: Recovery Room: two of three shares reconstruct the key and decrypt the vault

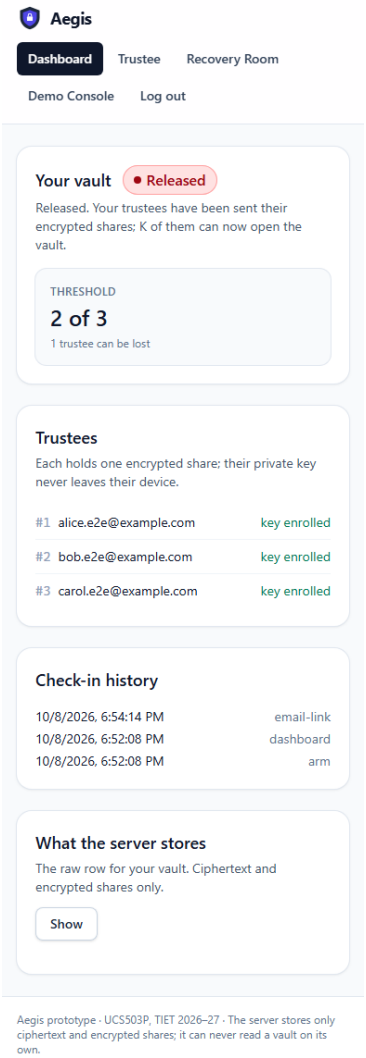


Figure 3.11: The dashboard on a phone-sized screen

3.3 Analysis

Did we meet our objectives? Largely, yes. The central safety objective was met: across 1,000 simulated schedules with restarts, racing and duplicate ticks and clock jitter, there were 0 early releases, 0 duplicate releases and 0 missed releases, and the same held with the slower deployment cadence. The zero-knowledge inspection found 0 occurrences of the plaintext, the key or any share in the stored data or the requests, including the database left by the real browser test. The threshold property was demonstrated end to end in the browser. Coverage on the crypto and scheduler packages (100.0% and 98.8%) exceeds the 80% target.

Where did we exceed expectations? Beyond the plan, the Python and TypeScript implementations were bound by shared known-answer vectors (13/13 matched), the scanner used for the zero-knowledge check carries its own positive-control self-test, and the whole demo script runs as an automated test. Locally, the sequential check-in p95 of 23.7 ms is well inside the 500 ms target, and on the live Vercel + Neon deployment it was 352.3 ms. Lighthouse scored Performance 100, Accessibility 100, Best practices 96, SEO 91 on the live landing page.

What challenges did we encounter, and which targets were missed?

- **Serverless scheduling.** Vercel functions keep no process alive and Hobby-plan cron jobs run at most daily, so the planned APScheduler design was replaced by an external trigger. With the 5-minute GitHub Actions trigger alone, the maximum release lateness was 809.195 s and the maximum prompt delay 828.542 s, so `NFR-REL-2` and `NFR-PERF-2` (both 60 s) are not met by the cron alone. They are met at a cadence of under a minute, which the lazy due-checks and the Demo Console provide during use and demonstration.
- **Email domain.** Without a verified sending domain, Resend delivers only to the account owner's address; the in-app Outbox (`EMAIL_MODE=log`) keeps the demonstration independent of email delivery.
- **Payload size.** Request bodies to Vercel functions are limited, so payloads are capped at about 1 MB and stored in the database; object storage for large files is future work.
- **Concurrency.** The load test exposed a stall: FastAPI runs a synchronous dependency's setup, the endpoint and the commit as separate thread-pool steps, so threads waiting for a database lock could starve the lock holder. 50 simultaneous check-ins on one vault still took 1079.0 ms at the 95th percentile locally because SQLite serialises writes.
- **Best-effort cron.** GitHub runs scheduled workflows on a best-effort basis and delays them under load; on this repository, consecutive scheduled runs of the 5-minute workflow were observed hours apart, so the cron-only lateness figures above are a lower bound for the live site.

- **Lighthouse deductions** are minor: the console logs the expected 401 from the logged-out session check (Best Practices), and `/robots.txt` is answered by the single-page-app rewrite instead of a real file (SEO).
- **Residual security risks** from the threat model remain: a malicious operator could serve altered client code (`T-4`) or withhold a release (`T-7`); K or more colluding trustees can open a vault by design (`T-2`); and metadata such as trustee emails and schedules is not encrypted.

How did we resolve them? The scheduler was redesigned around a pure decision function and an idempotent, transactional tick with several independent triggers; exactly-once delivery is enforced by unique deduplication keys in the outbox; the thread-pool stall was removed by limiting in-flight requests below the pool size; and an invalid package name inherited from the course template, which broke Vercel's dependency installation, was corrected. Each fix was confirmed by re-running the relevant measurement.

Chapter 4

Conclusion

4.1 Summary of Work

The team built a working prototype of Aegis, an encrypted dead-man's-switch vault in which the browser encrypts a secret, splits its key with Shamir's Secret Sharing and wraps each share for a trustee, and the server only keeps time and stores ciphertext. The prototype covers the full demonstration script — registration, trustee enrolment with browser-generated keys, sealing, one-click check-in, the Active, Warning, Grace and Released lifecycle, share decryption by trustees and K-of-N recovery — and is accompanied by a measurement suite whose results are reported in Chapter 3.

4.2 Key Findings

- **Finding 1:** A pure decision function plus a transactional, idempotent tick made the never-early and exactly-once properties testable at scale: 0 early and 0 duplicate releases over 1,000 fault-injected schedules.
- **Finding 2:** A zero-knowledge server is practical with standard browser cryptography: the inspection found 0 secrets on the server, and client-side costs are small (AES-256-GCM over 1 MiB in a median of 0.922 ms).
- **Finding 3:** On serverless hosting, timeliness depends on the tick trigger rather than on the code: the same scheduler meets the 60-second targets at a sub-minute cadence but not with a 5-minute cron alone.

4.3 Future Work and Recommendations

1. **Enhancements:** multiple vaults per owner, editing a vault after creation, account recovery, two-factor authentication, trustee-side reminders and an audit export.
2. **Alternative approaches:** verifiable secret sharing so that tampered shares are rejected (threat `T-9`), constant-time field arithmetic, and a signed or extension-delivered client to remove the code-delivery risk (threat `T-4`).
3. **Scalability:** object storage for files larger than 1 MB, Alembic migrations, rate limiting and a dedicated scheduler (or a paid cron) to meet `NFR-REL-2` without relying on lazy due-checks.
4. **Integration:** a production email domain with Resend, and optional notification channels behind the existing `EmailSender` strategy (SMS remains out of scope

under `NG-1`).

5. **Real-world deployment:** an independent security review, an admin interface, monitoring of the tick triggers, and backup and retention policies for the encrypted store.

4.4 Final Remarks

Building the prototype taught us to treat the demonstration as an acceptance test: once the five-minute demo script ran as an automated browser test, every change was checked against what we would actually present. We also learned to measure rather than estimate. Every number in this report comes from a script in the repository, including the two timeliness targets that the cron trigger alone does not meet. Finally, serverless hosting changed the design more than we expected: with no long-running process, the scheduler became a pure function triggered from outside, which turned out to be easier to test and to reason about.

Bibliography

[Bir+21] Alex Biryukov, Daniel Dinu, Dmitry Khovratovich, and Simon Josefsson. *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. RFC 9106, Internet Engineering Task Force. 2021. DOI: 10.17487/RFC9106 (cit. on p. 14).

[Dwo07] Morris J. Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D. National Institute of Standards and Technology, 2007. DOI: 10.6028/NIST.SP.800-38D (cit. on p. 9).

[Met] Meta Open Source. *React Documentation*. https://react.dev/. Accessed 2026-10-08 (cit. on p. 9).

[MH+19] David R. MacIver, Zac Hatfield-Dodds, et al. "Hypothesis: A New Approach to Property-Based Testing". In: *Journal of Open Source Software* 4.43 (2019), p. 1891. DOI: 10.21105/joss.01891 (cit. on pp. 14, 19).

[Mic] Microsoft. *Playwright Documentation*. https://playwright.dev/. Accessed 2026-10-08 (cit. on p. 9).

[Mor+16] Kathleen Moriarty, Burt Kaliski, Jakob Jonsson, and Andreas Rusch. *PKCS #1: RSA Cryptography Specifications Version 2.2*. RFC 8017, Internet Engineering Task Force. 2016. DOI: 10.17487/RFC8017 (cit. on p. 9).

[PM20] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. 9th. McGraw-Hill Education, 2020 (cit. on p. 17).

[Ram] Sebastián Ramírez. *FastAPI Documentation*. https://fastapi.tiangolo.com/. Accessed 2026-10-08 (cit. on p. 7).

[Sha79] Adi Shamir. "How to Share a Secret". In: *Communications of the ACM* 22.11 (1979), pp. 612–613. DOI: 10.1145/359168.359176 (cit. on pp. 5, 15).

[SQL] SQLAlchemy Authors. *SQLAlchemy 2.0 Documentation*. https://docs.sqlalchemy.org/en/20/. Accessed 2026-10-08 (cit. on p. 7).

[Ver] Vercel. *Using the Python Runtime with Vercel Functions*. https://vercel.com/docs/functions/runtimes/python. Accessed 2026-10-08 (cit. on p. 7).

[Wat17] Mark Watson. *Web Cryptography API*. W3C Recommendation. 2017. URL: https://www.w3.org/TR/WebCryptoAPI/ (cit. on p. 9).